

Datapath

Metodologia di clocking

Singolo ciclo: ciclo singolo di lunghezza fissa uguale al tempo necessario per eseguire l'istruzione più lunga. Ogni istruzione viene eseguita in 1 ciclo di clock. Alcune unità funzionali vengono replicate.

Svantaggi: spreco di tempo (istruzioni che richiederebbero meno tempo sono rallentate)

Multiciclo: ogni istruzione viene eseguita in più colpi di clock, quindi istruzioni di tipo diverso vengono eseguite in cicli di clock diversi. Le unità funzionali vengono usate più volte durante l'esecuzione della stessa istruzione in cicli di clock differenti quindi c'è meno replicazione.

Svantaggi: complessità, la gestione del clock è compito dell'unità di controllo

Unità funzionali

1. Memoria:

- Contiene istruzioni e dati (nel datapath multiciclo c'è un'unica memoria, mentre nella soluzione a singolo ciclo istruzioni e dati si trovano su memorie diverse);
- Sulla base dell'indirizzo in input, restituisce in output l'istruzione o il dato letto

2. Register file:

32 registri che contengono i dati utilizzati nel corso dell'esecuzione delle istruzioni;

Restituisce in output il contenuto dei due registri letti (indicati dai due input ReadRegister1 e ReadRegister2) e scrive, se il segnale di scrittura è attivo, il dato in input WriteData nel registro indicato dall'indirizzo in input (WriteRegister).

3. ALU: per tutte le istruzioni:

- Incrementa il PC ($PC+4$) e calcola il valore di branch (che sarà usato solo nel caso di istruzioni di branch)
- **Istruzioni R-type:** esegue operazioni aritmetico-logiche su due operandi in funzione del function-code
- **Istruzioni ad accesso a memoria:** calcola l'indirizzo di memoria cui accedere
- **Istruzioni branch:** confronta i due registri in input

4. Sign Extended: opera l'estensione di segno dai 16 bit in input ai 32 bit in output

5. Shift Left 2: opera uno shift a sinistra dei bit in input (con l'effetto di moltiplicare per 4 l'input)

Registri:

Al termine di ogni ciclo di clock i valori intermedi vengono memorizzati nei registri addizionali e restano disponibili per il ciclo successivo.

1. **PC:** contiene i 32 bit che indicano l'indirizzo dell'istruzione da eseguire
2. **IR:** Instruction register: contiene i 32 bit che corrispondono alla codifica dell'istruzione prelevata da memoria per l'esecuzione
3. **Memory Data Register:** contiene il dato letto da memoria prima della sua scrittura nel registro destinazione (nel caso di esecuzione di istruzione load)
4. **A e B:** contengono la copia dei valori dei 2 registri operandi (tra il Register File e la ALU)
5. **ALUOut:** contiene la copia dell'output della ALU

1) Si spieghi perché il circuito "register file" fornisce in uscita il contenuto di due suoi registri contemporaneamente

Perché nel Datapath Multicycle i due registri in uscita dal register file finiscono nell'ALU e così è possibile eseguire operazioni aritmetiche tra due registri.

Kernel

2) Si descriva la necessità di avere due modi di funzionamento del processore (chiamati user e kernel in molti processori) per garantire la coesistenza di spazi di indirizzamento diversi.

Il codice macchina eseguito in modalità kernel ha un accesso illimitato alla memoria, all'hardware e alle altre risorse. Al contrario in user mode il sistema ha un accesso limitato.

3) Si descrivano i vantaggi e gli svantaggi di avere uno stack separato dedicato al kernel:

Vantaggi: la possibilità di memorizzare dati importanti come il gestore eccezioni, all'interno di uno spazio di indirizzamento dedicato e non accessibile da parte del programma utente.

Svantaggi: la memoria è limitata a causa del fatto che va condivisa tra due diversi stack.

4) Si descrivano i vantaggi e gli svantaggi di avere più di due registri dedicati al kernel:

Nell'architettura MIPS 32 troviamo 2 registri dedicati al kernel: \$k0 e \$k1. Il vantaggio di averne ulteriori consiste nel fatto che, in modalità kernel, si potranno usare più registri senza la preoccupazione di doverli salvare per poi ripristinarli ai loro valori originali nel momento in cui il controllo torna allo spazio dedicato all'utente. Lo svantaggio principale è quello di perdere ulteriori registri dedicati alla modalità utente. Infatti, avendo il limite imposto di 32 registri, e considerando che ce ne sono altri, oltre a \$k0-1, dedicati a specifiche funzioni, ci ritroveremmo con pochi registri per svolgere le normali funzioni del programma utente.

5) Quali sono i principali tipi di segnali che si scambiano datapath e unità di controllo e a che cosa servono:

I segnali che si scambiano sono i segnali di controllo e i segnali di stato. I segnali di controllo servono per controllare appunto le selezioni di tutti i componenti usati nel datapath; mentre quelli di stato servono per capire cosa dovrà essere eseguito dopo.

6) Si descrivano i vantaggi e gli svantaggi di avere più gestori delle eccezioni, a indirizzi predeterminati, dedicati a specifiche classi di eccezioni o interrupt:

Esistono due metodi per gestire le eccezioni: il primo consiste nell'avere un programma dedicato alla gestione di esse situato da una precisa locazione di memoria in poi; il secondo consiste nelle eccezioni vettorizzate: questo meccanismo si basa sul fare riferimento a specifici indirizzi di memoria che contengono diversi gestori delle eccezioni, ognuno specializzato per gestirne un particolare tipo. Il vantaggio è quello di non necessitare di un registro Cause, in quanto l'indirizzo a cui saltare è definito dal tipo di eccezione che si presenta.

Fasi per eseguire un'istruzione

1. Fetch

Ha lo stesso comportamento per tutti i tipi di istruzione.

Legge l'istruzione dalla memoria e la salva nel registro IR.

Dopo la lettura, il PC viene incrementato di 4 per puntare alla prossima istruzione da leggere

2. Decode

- 1) Lettura dei vari campi dell'istruzione caricata in IR, il processore identifica il tipo di istruzione da eseguire verificando opcode e function code se necessario
- 2) Lettura dei registri dal Register-file e copia dei bit (25-21) nel registro A e (20-16) nel registro B.
- 3) Nel caso di una branch si calcola l'indirizzo a cui saltare ($PC + \text{sign-extend} + \text{shift left } 2$)

3. Execute:

Ha comportamenti diversi ed è composta da un diverso numero di passi in base al tipo dell'istruzione.

Nel caso di R-type:

l'execute è composta da 2 passaggi

- 1) L'ALU esegue l'operazione aritmetico-logica tra i valori dei registri A e B e copia il risultato nel registro ALUOut
- 2) Nel ciclo successivo, il Register-file scrive nel registro destinazione (identificato dai bit (15-11)) il valore di ALUOut

Nel caso di J-type:

l'execute è composta da 1 passaggio

- 1) Aggiornamento PC: non si utilizza la ALU, inserimento nei bit più significativi i 4 bit più significativi del PC, aggiungere i 26 bit del jump address e moltiplicarli*4 (shiftare a sinistra di 2 bit)

Nel caso di branch:

l'execute è composta da 1 passaggio

- 1) L'ALU confronta i valori dei due registri A e B e se il confronto dà esito positivo $PC = ALUOut$

Nel caso di sw (store word):

l'execute è composta da 2 passaggi

- 1) L'ALU calcola la somma tra l'offset e il contenuto del registro sorgente (tenendo in considerazione il segno)
- 2) Salvataggio nel registro B il contenuto del registro ALUOut

Nel caso di lw (load word):

l'execute è composta da 3 passaggi

- 1) L'ALU calcola la somma tra l'offset e il contenuto del registro sorgente (tenendo in considerazione il segno)
- 2) Salvataggio nel registro MDR il contenuto del registro ALUOut
- 3) Nel ciclo successivo, il Register-file scrive nel registro destinazione (identificato dai bit (20-16)) il valore di MDR

Eccezioni: eventi involontari

Variazioni rispetto alle condizioni normali di funzionamento della CPU.

Eccezioni: evento sincrono generato all'interno del processore e provocato da problemi nell'esecuzione di un'istruzione. esempi: overflow, opcode non valido ecc.. Se l'eccezione è risolvibile, allora il programma riprende la sua esecuzione, altrimenti termina prima della fine.

Interrupt: evento asincrono che non dipende dall'esecuzione del programma in corso, ma da un segnale da parte di un hardware esterno al processore (di solito arriva dalle periferiche di I/O). Questo segnale è proprio detto interrupt ed è utilizzato dalle periferiche per segnalare alla CPU il verificarsi di certi eventi. Queste, sono gestite tra 2 istruzioni consecutive (nessuna istruzione verrà mai interrotta nelle sue fasi di fetch, decode ed execute). Si sospende l'esecuzione del programma utente, si gestisce l'interruzione e poi si riprende regolarmente l'esecuzione

Conseguenze: cambiamento del normale flusso di esecuzione di un programma.

In tutti i casi (in MIPS), il flusso di esecuzione passa ad una locazione di memoria prefissata (0x8000 0180), dove si trova il gestore delle eccezioni (exception handler).

Gestione delle eccezioni

Tutti i processori eseguono i seguenti passi per gestire un eccezione/interruzione:

1. Interruzione dell'esecuzione del programma corrente
2. Salvataggio dell'indirizzo dell'istruzione che ha causato l'eccezione (in EPC) quindi PC-4 per le eccezioni, PC nel caso di interrupt.
3. Salvataggio parziale dello stato del programma per poter poi ripristinare, gestita l'eccezione, tutti i registri per il continuo dell'esecuzione.
4. Aggiornamento di PC con l'indirizzo 0x80000180
5. Esecuzione della procedura di gestore delle eccezioni – exception handler (collocata nello spazio kernel text a partire dalla locazione 0x80000180)
6. Se possibile, ripristino dello stato del programma al momento dell'eccezione e continuazione del regolare svolgimento.

Il datapath predispone correttamente alcuni registri in modo da segnalare e rendere visibile cosa è successo al software di gestione.

Il registro cause contiene il valore che identifica la causa del verificarsi dell'eccezione. Ogni eccezione ha un codice di 5 bit che la identifica (pag 34 dell'appendice A)

Exception Handler:

- Porzione di codice assembly che contiene il codice che gestisce le eccezioni
- Si trova alla locazione fissa 0x80000180 in area ktext riservata al sistema operativo
- Dal momento che viene eseguito dopo l'interruzione di un programma utente in un punto qualsiasi, è tenuto a preservare TUTTI i registri macchina
- Ad esso sono riservati due registri \$k0, \$k1 che può utilizzare senza doverne preservare i contenuti
- Al termine del trattamento dell'eccezione, l'exception handler ritorna il controllo al programma interrotto (istruzione eret)

Se l'eccezione corrente era una interruzione, dopo la sua gestione, l'esecuzione del programma deve riprendere dall'istruzione corrente (salvata nel registro EPC)

Se si tratta di un altro tipo di eccezione ed è possibile proseguire (se non è avvenuto un errore irreparabile¹), l'esecuzione deve riprendere dall'istruzione successiva (EPC+4)

Con salvataggio parziale si intende che non viene salvato tutto lo stato della CPU prima che venga gestita l'eccezione ma che si salvi solo quello che sarà modificato dall' interrupt service routine.

Interrupt: Quando le periferiche sono pronte a effettuare operazioni impostano una linea di interruzione collegata alla CPU. Quando una di queste viene attivata, se il sistema ha abilitato le interruzioni della CPU il normale flusso di esecuzione viene interrotto e si avvia la procedura di gestione dell'eccezione.

Registro status:

Bit 0 è dedicato all' interrupt enable

- 0- interruzioni non abilitate, e anche se la linea di interruzione è attivata dalla periferica, la CPU non genera l'eccezione
- 1- interruzioni abilitate e la periferica che solleva una eccezione può essere «percepita»

I bit (15-8) contengono la interrupt mask

6 bit dedicati alle 6 linee delle periferiche e 2 bit per gestire l' interrupt a livello software

Quando viene sollevata una eccezione a seguito di un'interruzione abilitata, nel Cause register viene impostato a 1 il bit corrispondente all'interruzione (e i 5 bit dell'exception code sono impostati a 0)

Cosa fa la cpu/sistema quando è concluso un interrupt (processo di restore)?

Il PC viene ripristinato al precedente valore (meno 4, visto che bisogna tornare all'istruzione di prima) grazie al registro EPC, i contenuti dei vari registri vengono ripristinati, i bit del registro cause vengono azzerati, Interrupt Enable viene rimesso a 1. Se si parla di periferiche, il bit ready viene rimesso a 0.

Dispositivi I/O

Bus comune che collega la CPU sia con la memoria che con le periferiche. È composto da:

1. **Bus dati:** trasferisce dati e istruzioni
2. **Bus controllo:** trasporta informazioni per la definizione delle operazioni da compiere e per la sincronizzazione tra i dispositivi
3. **Bus indirizzi:** la CPU trasmette gli indirizzi di memoria o di periferica che identificano i dati da leggere/scrivere dalla memoria/periferiche

Periferiche mappate in memoria

Ogni periferica ha uno spazio dedicato nella memoria e ad ognuna di esse viene assegnato un ID univoco. I registri dell'interfaccia della periferica sono mappati in memoria, questi non sono accessibili da un programma utente. Le periferiche sono collegate alla CPU in primo luogo tramite un'interfaccia, questa è collegata al bus che permette la comunicazione. L'interfaccia contiene:

1. **Registro dati:** rappresenta i dati di input o di output in base alla periferica
2. **Registro di stato:** rappresenta lo stato della periferica e viene letto dalla CPU indica se abilitare la generazione di interrupt e contiene un bit ready (che indica se la periferica è pronta (nel caso di una tastiera indica se un dato è pronto per essere prelevato))

Prestazioni:

Banda passante: quantità di dati che si può trasferire per unità di tempo $\frac{\text{bit o byte}}{\text{secondo}}$

Latenza: rappresenta il tempo che intercorre tra l'istante in cui la periferica è pronta per il trasferimento dei dati e l'istante in cui il dato viene trasferito

Tecniche di gestione I/O

1. Programmed I/O (gestito a controllo di programma)

In questa soluzione la CPU è dedicata solo a interrogare la periferica, questa restituisce il suo stato e se è ready la periferica è pronta a trasferire/ricevere dati.

La CPU si occupa del trasferimento dei dati. Il cambiamento del bit ready da 0 a 1 dopo il prelevamento del dato è compito della periferica

Se la periferica è lenta a rispondere (e di fatto lo è), il processore è in "busy wait", questo termine indica il tempo che il processore spreca a interrogare una periferica non ancora pronta, quando potrebbe andare a svolgere altri compiti.

Vantaggi: risposta veloce al bit ready

Svantaggi: CPU bloccata in stato "busy wait"

Prestazioni:

- 1) Banda passante: alta (la CPU trasferisce subito il dato e la gestione delle periferiche richiede poche istruzioni e quindi poco tempo)
- 2) Latenza: minima (la CPU nota subito lo stato della periferica perché fa solo quello)

2. Interrupt driven I/O (guidato da interrupt)

In questa soluzione il cambiamento di stato del ready bit è controllato dalla periferica, non più dalla CPU che nel frattempo può occuparsi di altri compiti e non rimane più nello stato di "busy wait".

- 1) Quando lo stato è ready la periferica avvisa la CPU inviando un segnale di interrupt request

- 2) Quando il processore se ne accorge informa la periferica con un segnale di interrupt acknowledge
- 3) La CPU interrompe l'esecuzione del programma corrente ed esegue la procedura di risposta all' interrupt
- 4) Terminata questa, la CPU riprende l'esecuzione del programma interrotto

Perché il controllo dello stato può essere fatto dalla periferica?

Grazie al meccanismo delle eccezioni: la periferica invia un interrupt, questo viene gestito dall' exception handler e poi avviene il trasferimento effettivo di dati

Vantaggi: CPU non più bloccata in stato "busy wait" risposta

Svantaggi: la gestione delle operazioni di trasferimento e la risposta a un interrupt è un lavoro molto oneroso per il processore

Prestazioni:

1. **Banda passante:** minore in quanto il trasferimento di ogni dato necessita di molto lavoro e istruzioni e quindi di un tempo molto elevato.
2. **Latenza:** maggiore (per la maggiore quantità di istruzioni e operazioni da eseguire)

3. DMA (accesso diretto alla memoria)

La periferica diventa autonoma nell' accesso alla memoria (non si deve più passare per il processore per la comunicazione tra le due). La periferica gestisce il trasferimento dei dati.

L' unico compito della CPU è quello di istruire la periferica con l'aggiunta di registri mappati in memoria oltre al registro dati e il registro stato. Il primo è un registro che indica l' indirizzo di memoria da/dove trasferire i dati. Il secondo è un registro che indica la quantità di dati da trasferire.

Alla fine del trasferimento la periferica invia un interrupt alla CPU per segnalare il completamento del trasferimento.

- 1) **Banda passante:** massima perché la CPU non deve eseguire nessuna istruzione
- 2) **Latenza:** minima dato che nessuna istruzione è eseguita dalla CPU

Pipeline

Modifica al datapath singolo ciclo che permette di avere in contemporanea più istruzioni in sezioni diverse del datapath. Questa soluzione si ottiene aggiungendo dei registri chiamati "pipeline registers" che permettono di tenere separate le varie sezioni del datapath durante lo stesso ciclo di clock. Quindi se osserviamo il datapath in un dato istante, vediamo attive diverse unità funzionali su istruzioni differenti. I registri permettono di tenere separate le informazioni e i dati relativi a un'istruzione dalle altre istruzioni che si trovano nel datapath (quindi ogni istruzione non potrà mai vedere i dati di altre istruzioni). Con pipeline abbiamo lo sfruttamento massimo contemporaneo di tutto l'hardware del datapath. I registri pipeline contengono la copia degli output della sezione a sinistra, per poi presentarli all' istante successivo alla sezione di destra.

Senza la pipeline il segnale fluiva con continuità da sinistra verso destra, ora il segnale può attraversare i pipeline registers solo se viene abilitata la scrittura e quindi l'aggiornamento di questi. Anche i segnali di controllo vengono memorizzati in appositi registri per allineare nel tempo il segnale che fluiva con i segnali di controllo. I segnali di controllo vengono consegnati alla sezione del datapath che li utilizza.

Hazard

Rischi di malfunzionamento, si verificano quando un'istruzione non può essere seguita immediatamente da un'altra. Esistono 3 tipi di hazard:

Structural hazard: dovuti alla struttura del datapath per esempio nella soluzione a singolo ciclo la memoria contiene sia istruzioni che dati, questo potrebbe generare hazard e per risolverli basta dividere memoria dati e memoria istruzioni come per il multiciclo.

Control hazard: interferenze date dal tentare di prendere decisioni prima che la condizione venga valutata

Data hazard: tentare di utilizzare dati prima che siano pronti, gli operandi sorgente di un'istruzione sono prodotti dall'istruzione precedente ancora in pipeline, istruzione di caricamento seguita immediatamente da un'istruzione aritmetico logica che utilizza l'operando di caricamento come valore di origine e non quello aggiornato ecc..

es: add r1, r4,r6 (qui si vuole aggiornare r1)

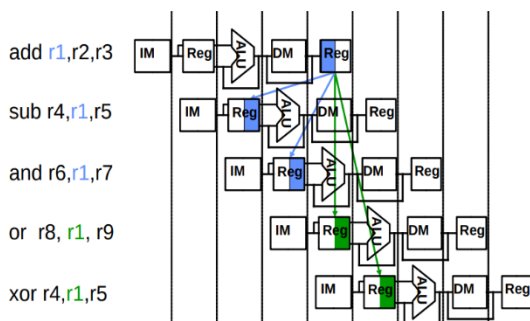
add r2,r1,r4 (qui si vuole leggere r1 già aggiornato) ma non passa abbastanza tempo per aggiornare il registro quindi qui si verifica un hazard

Soluzione 1: STALLO/NOP (no operation)

Se mi trovo nella situazione in cui un'istruzione cerca di accedere ad un registro aggiornato nelle istruzioni precedenti, blocco l'entrata in pipeline di certe istruzioni fino a quando il dato non sarà stato aggiornato e pronto per essere letto da quell'istruzione. Si introduce così una situazione di stallo, un congelamento del normale flusso del datapath con pipeline con il conseguente spreco di sezioni datapath disponibili. Con questa soluzione gli hazard si possono sempre risolvere

Soluzione 2: DIVISIONE CLOCK

Questa soluzione propone di dividere il clock in 2 parti, durante il primo fronte scrivo e copio il valore aggiornato di un registro nel registro di transito e durante il secondo fronte leggo il valore appena aggiornato. Non è sempre possibile risolvere gli hazard con questa soluzione a causa della vicinanza tra le istruzioni, e il tempo che ci impiega ognuna di esse ad aggiornare i dati che serviranno poi alle istruzioni successive.



Per le istruzioni sub e and non è possibile prelevare il dato aggiornato in fase di decode perché non passa abbastanza tempo dall'istruzione che aggiorna il registro r1 alla possibilità di leggere il valore aggiornato, quindi per queste istruzioni l'hazard non verrà risolto. Mentre per or e xor la soluzione della divisione del clock risolve gli hazard. Durante la prima parte del quinto colpo di clock l'istruzione add aggiorna il valore di r1 e nello stesso istante ma nella sua seconda parte l'istruzione or preleva il valore del registro aggiornato e così si risolve l'hazard.

Soluzione 3: FORWARDING

Predisporre con dei collegamenti la disponibilità di dati diretta a fasi successive. Quando il dato esce dall'ALU lo rende disponibile all'istante successivo come operando dell'ALU stessa per l'istruzione successiva. Non sono i valori aggiornati dei registri ad essere operandi, ma i valori

risultanti dall'operazione della ALU precedente evitando così di aspettare di compiere l'operazione di write back. Con questa soluzione è necessaria una modifica del datapath.

lw r1, 100(r2)

and r6, r1, r7

il confronto tra r1 e r7 non lo faccio con il valore aggiornato del registro r1 ma con ciò che prelevo dalla memoria con l'istruzione di load word

Soluzioni per risolvere i control hazard

I control hazard si verificano tipicamente quando ho a che fare con istruzioni jump o branch.

Una soluzione possibile è quella di introdurre una situazione di stallo.

Nelle istruzioni di salto bisogna aggiornare il contenuto del PC per poter puntare e saltare all'istruzione desiderata. Questa operazione nella branch impiega più tempo perché oltre a saltare ad un indirizzo bisogna anche confrontare il contenuto di 2 registri.

Con la soluzione dello stallo, posso sempre risolvere i control hazard.

Una seconda tecnica è quella di "predire" l'esito del confronto tra i valori dei registri nella branch: aggiorni il PC assumendo che il controllo sia sempre fallito. Se il controllo dovesse risultare vero, devo cancellare tutte le istruzioni prima del salto che non dovevano essere seguite e i registri eventualmente utilizzati da queste devono essere ripristinati.

Quindi i 2 problemi principali che posso riscontrare con i control hazard sono:

1. per le istruzioni branch: rischio di accedere a registri non ancora aggiornati
2. produrre un aggiornamento del PC che va a invalidare l'istruzione successiva (non più puntata da PC). la conseguenza è quella di avere un errato inserimento in pipeline delle istruzioni successive alla jump o alla branch che interrompono la normale sequenza temporale di entrata in pipeline)

Gerarchia di memoria

Consiste in un insieme di livelli di memoria, ciascuno caratterizzato da una diversa velocità, dimensione, costo e funzionalità differenti. A parità di capacità, le memorie più veloci hanno un costo più elevato per singolo bit di quelle più lente.

Un livello di memoria più vicino al processore contiene un sottoinsieme di dati memorizzati in ogni livello sottostante e tutti i dati si trovano nel livello più basso

Le memorie cache si trovano nella parte alta della piramide della gerarchia: (tra la CPU e le memorie centrali) sono più piccole, più veloci e più costose ma di limitate dimensioni, infatti al crescere della distanza dalla CPU diminuisce il costo, ma aumenta la capacità di memorizzazione.

La memoria centrale è molto capiente ma ha tempi di accesso più elevati. È accessibile in modo diretto tramite indirizzi.

PRINCIPIO DI LOCALITÀ!

Viene sfruttato strutturando la memoria in modo gerarchico in base alla:

1. **Velocità** (più è veloce, più è vicina al processore)
2. **Dimensione** (più è grande, più è lontana dal processore, meno costosa)

Esistono due tipi di località:

1. **Temporale:** quando si fa riferimento a un elemento, c'è la tendenza a fare riferimento allo stesso elemento anche negli istanti immediatamente successivi
2. **Spaziale:** quando si fa riferimento a un elemento c'è la tendenza a fare riferimento poco dopo ad altri elementi che hanno l'indirizzo vicino ad esso (adiacenti)

Definizioni

Blocco/linea: la più piccola quantità di informazione che può essere presente/assente in una gerarchia di memoria

- **Hit (successo nell'accesso):** l'informazione richiesta dal processore è valida e si trova in uno dei blocchi nel livello superiore di memoria.
- **Miss (fallimento nell'accesso):** il dato non è presente nel livello immediatamente superiore ed occorre accedere al livello più distante oppure non è valido.
- **Hit rate (frequenza di hit):** frazione degli accessi alla memoria nei quali l'informazione richiesta è stata trovata nel livello superiore di memoria.
$$\frac{\text{Accessi risolti con successo}}{\text{Numero totale di accessi}}$$
- **Miss rate (frequenza di miss):** frazione degli accessi alla memoria nei quali l'informazione richiesta non è stata trovata nel livello superiore di memoria $(1 - \text{HitRate})$
$$\frac{\text{Accessi risolti senza successo}}{\text{Numero totale di accessi}}$$
- **Tempo di hit:** tempo di accesso al livello superiore della memoria, comprende anche il tempo necessario a stabilire se il tempo di accesso si risolva in un successo o in un fallimento
- **Tempo di miss:** il tempo necessario a sostituire un blocco del livello superiore con un nuovo blocco dal livello inferiore della gerarchia, e trasferire i dati di questo blocco al processore
- **Remind:** una gerarchia di memoria può essere composta da più livelli, ma i dati vengono trasferiti solo tra due livelli vicini

Cache:

L'algoritmo di caching si basa sui principi di località spaziale e temporale:

- Mantiene i dati richiesti recentemente “vicino” alla CPU (località temporale)
- Muove blocchi contigui di memoria che contendono il dato richiesto (località spaziale)

Cache a mappatura diretta:

Associa una sola locazione della cache a ogni parola della memoria definendo una corrispondenza tra l'indirizzo in memoria della parola e la locazione nella cache. A una linea di cache possono corrispondere più blocchi all'interno della memoria.

L'indirizzo di memoria è formato da un:

- **Tag (etichetta):** contiene informazioni necessarie a verificare se una parola della cache corrisponde o meno alla parola cercata.
- **Indice:** utilizzato per selezionare il blocco della cache
- **Offset:** bit necessari per selezionare il byte richiesto nella parola

Dato che il numero di blocchi nella cache è una potenza di 2, la posizione corrispondente della parola in cache è data dai $\log_2(\text{numero elementi nella cache})$ bits meno significativi dell'indirizzo in memoria principale.

Ogni linea di cache include:

Bit di validità: indica se i dati presenti nella linea di cache sono validi, all'avvio del calcolatore, tutte le linee sono di default non valide

- **Tag (etichetta):** consente di individuare in modo univoco il blocco in memoria che è stato mappato in linea di cache

- **Blocco di dati:** vero e proprio formato da una o più parole

Indirizzo: **00101** tag indice

Consideriamo una cache con 64 blocchi di 16 byte ciascuno. L'indirizzo 1200 può essere ricavato come: (indirizzo blocco) modulo (numero blocchi cache)

Indirizzo blocco: $1200/16=75 \text{ modulo } 64= 11$

Una cache con blocchi di dimensioni maggiori sfruttano maggiormente la località spaziale diminuendo la MISS PENALTY, questa però torna a crescere se la dimensione dei blocchi diventa troppo grande rispetto alla dimensione della cache

MISS PENALTY = differenza tra il tempo di accesso al livello inferiore e il tempo di accesso alla cache

Cache full associative: corrispondenza 1 a 1 tra i blocchi di memoria e la cache, ogni blocco può essere collocato in qualsiasi locazione della cache. Non ho conflitti ma per ricercare un blocco nella cache è necessario cercarlo in tutte le linee della cache (soluzione molto lenta)

Cache Set Associative: soluzione intermedia tra direct mapped e fully associative

I blocchi appartenenti alla cache sono raggruppati in set

In una cache set associative ad N vie (N-way set associative) ogni set raggruppa N blocchi

Ogni indirizzo di memoria corrisponde ad un unico set della cache (accesso diretto tramite indice) e può essere ospitato in un blocco qualunque appartenente a quel set

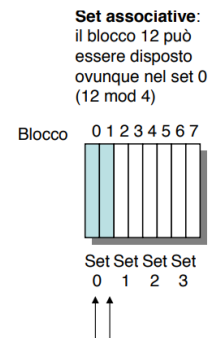
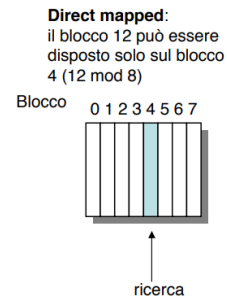
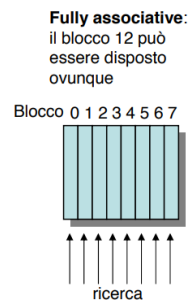
Stabilito il set, per determinare se un certo indirizzo è presente in un blocco del set è necessario confrontare in parallelo i tag di tutti i blocchi

Cache N-way set associative a confronto con cache ad accesso diretto:

- N comparatori invece di 1
- ulteriore ritardo fornito dal multiplexer
- il blocco è disponibile dopo la decisione Hit/Miss e la selezione del set
- In una cache ad accesso diretto, il blocco è disponibile prima della decisione Hit/Miss

Incremento dell'associatività

- Diminuzione del miss rate
- Maggior costo implementativo



Algoritmi per la sostituzione di blocchi

- Nella cache ad accesso diretto: se il blocco di memoria è mappato in una linea di cache già occupata (conflict miss), si elimina il contenuto precedente della linea e si rimpiazza con il nuovo blocco
- In caso di cache completamente associativa: ogni blocco è un potenziale candidato per la sostituzione
- In caso di cache set-associativa a N vie: bisogna scegliere tra gli N blocchi del set

Politiche di sostituzione

1. Politica di sostituzione **Random** – Scelta casuale (molto facile da implementare a livello hardware)
2. Politica di sostituzione **Least Recently Used (LRU)** sfruttando la località temporale, il blocco sostituito è quello che non si utilizza da più tempo

Ad ogni blocco si associa un contatore all'indietro, che viene portato al valore massimo in caso di accesso e decrementato di 1 ogni volta che si accede ad un altro blocco

3. Politica di sostituzione First In First Out (**FIFO**): si seleziona il blocco più vecchio anziché quello non usato da più tempo

Gestione dei miss in lettura

Se un blocco non è presente nella cache bisogna mettere in stallo l'intera CPU. Un'unità di controllo andrà a prelevare un blocco dal livello inferiore della gerarchia e lo trasferisce all'interno della cache.

In generale al verificarsi di un miss nella cache delle istruzioni sono necessari i seguenti passi:

1. Inviare PC-4 alla memoria
2. Lettura dalla memoria
3. Scrittura nella cache (dato, tag e bit di validità)
4. Riavviare l'esecuzione dell'istruzione che ha causato il miss.

l'operazione può essere effettuata ancora prima di verificare il match dei tag (per velocizzare)

Gestione dei miss in scrittura

Scrivere un dato nella cache significa creare un'incoerenza, se non si aggiornano i livelli inferiori della gerarchia di memorie.

Tale aggiornamento richiede lo stallo della CPU. Due tecniche risolutive:

- **Write-through:** i dati sono scritti nel blocco della cache e nel blocco del livello inferiore.

Vantaggi: è la soluzione più semplice da implementare e si mantiene la coerenza delle informazioni nella gerarchia di memorie

Svantaggi: le operazioni di scrittura vengono effettuate alla velocità della memoria di livello inferiore → diminuiscono le prestazioni.

- **Write-back:** i dati sono scritti solo nel blocco della cache.

Il blocco modificato viene scritto nel livello inferiore della gerarchia solo quando deve essere sostituito. Subito dopo la scrittura, cache e memoria di livello inferiore sono inconsistenti. Viene mantenuto un dirty bit (rappresenta lo stato del blocco della cache che può essere clean (non modificato) oppure dirty(modificato))

Vantaggi: le scritture avvengono alla velocità della cache, le scritture successive sullo stesso blocco alterano solo la cache

- **Svantaggi:** ogni sostituzione del blocco può provocare un trasferimento in memoria
- Utilizzo di un **write buffer**

Write buffer interposto tra la cache e la memoria di livello inferiore

- Il processore scrive i dati sia nella cache e nel write buffer
- Il controller della memoria scrive il contenuto del write buffer in memoria
- Efficiente se la frequenza di scrittura $\ll 1/\text{write cycle della DRAM}$
- Altrimenti, il buffer può andare in saturazione ed il processore deve aspettare che le scritture giungano a completamento (write stall)

Write miss: potrebbe accadere che cerchiamo di scrivere in un indirizzo che non è contenuto in cache. Soluzioni possibili:

- **Write allocate:** il blocco viene caricato in cache e si effettua la scrittura, combinazione con Write Back
- **No-write allocate:** il blocco viene scritto direttamente nella memoria di livello inferiore, senza essere trasferito in cache, combinazione con Write through

1) Vantaggi e svantaggi di una cache di grandi dimensioni:

Una cache molto grande consente di velocizzare l'esecuzione di un programma in quanto vengono diminuiti gli accessi alla memoria centrale (lenta) e aumentano gli accessi alla cache (veloce). Con un'ipotetica cache di dimensioni pari alla memoria centrale è possibile salvare tutto in memoria cache portando quindi l'hit rate al massimo. Lo svantaggio è che al crescere della memoria cache aumentano i costi.

2) Si descriva il funzionamento di una gerarchia che preveda più livelli di cache:

Con l'utilizzo di più livelli di cache è possibile ridurre notevolmente il tempo sprecato nel caso in cui si ottenga un miss sul primo livello di cache. Nel caso di cache a singolo livello, se si verifica un miss è sempre necessario andare a ricercare il dato sulla memoria centrale perdendo un numero di cicli di clock pari al miss penalty della cache. Nel caso di cache a più livelli se si verificasse un miss sulla prima cache potrebbe però verificarsi un hit sulle cache successive riducendo notevolmente i tempi con un comunque considerevole aumento dei costi e del miss penalty

Tecniche di gestione I/O

1) Si descriva brevemente quali bit del registro di stato di una periferica siano usati in un trasferimento a controllo di programma e si indichi la loro funzione:

Sono usati ready bit ed error bit. Il primo indica se la periferica è pronta per un nuovo trasferimento ed il secondo se si sono verificati errori.

2) Che bit si usano nel controllo di programma? Si usa il bit ready. La CPU continua ad interrogare la periferica per vedere se è pronta e una volta che il bit ready è uguale ad 1 ha inizio il trasferimento; una volta che il trasferimento è finito riprende il ciclo.

3) Perché dopo un trasferimento DMA la periferica fa una richiesta di interrupt? / Ha senso ipotizzare una architettura I/O che utilizzi esclusivamente il DMA e non preveda la possibilità di interrupt?

Per notificare che il trasferimento dati è concluso e scatenare una commutazione di stato. Se non ci fosse il trasferimento sarebbe invisibile al programma. Così invece la CPU prende i dati finiti e li consegna a chi li aveva chiesti. No, in questo modo la CPU non potrebbe sapere quando il trasferimento dati è completato.

1) Spiegare perché non tutte le funzioni di scrittura di un registro richiedono un componente e una linea di controllo:

Preso come esempio il register file, questo necessita di un controllo in scrittura perché non è sempre richiesta una modifica, quindi si usa il controllo write che può assumere valore 0 o 1 reciprocamente per non scrivere e scrivere; invece esistono registri che necessitano sempre e comunque di scrittura (ad ogni inizio di operazione) per esempio il PC che quindi non ha bisogno di controllo in scrittura.